Index Cards: A Deterministic Entity Index as Memory for Coding Agents, Measured Against Grep, Retrieval and Agent-Written Memory

Abhay Mettu
https://github.com/abhaymettu/index-cards

November 2026

Abstract

Coding agents increasingly answer questions from a user's own notes: who someone is, which project they own, what the deadline is now. We ask which memory layer gives the best answers when the agent may read only a fixed number of tokens. We build a benchmark from seeded, fictional six-month markdown vaults (251 notes, about 114k tokens each) whose questions have known answers, including facts that later changed and questions that need two notes. At a 4,000-token budget, with the same answering model and prompt for every system, a deterministic entity index (one card per person and project listing every dated mention, built by code with no model calls) answers 89.5% of held-out questions on a world nothing was tuned on. Embedding retrieval answers 82.1% (paired bootstrap $p = 0.0027$), hybrid BM25 plus embeddings 79.8%, BM25 77.0%, a Claude Code style memory file the model kept itself 63.0%, and grep 73.5%. The gain comes from questions that need two notes (70% against 31% for embeddings), and it holds from 2,000 to 4,000 tokens; at 1,000 tokens embeddings are ahead, and at 8,000 the two converge. Two findings run against common assumptions. Stale-fact errors are rare for every retriever once chunks carry dates; misses are mostly retrieval failures answered "unknown". And in an unbudgeted agent loop, where the model greps the vault itself, the index does not help: Sonnet scores the same without it at fewer tokens, and Haiku scores lower with it. Code, data, cached answers and the scorer are public, and every table rescores without model calls.

1 Introduction

A coding agent that works with one person for months needs what that person knows: names, projects, decisions, and which of them changed. Agent memory today is mostly one of two designs. The model writes its own memory file (Claude Code's auto memory, ChatGPT memory), or a retriever returns chunks from a store by similarity. A memory the model writes is only as current as the last time it chose to edit it. A similarity search returns whatever is worded most like the question, which is not always what the question needs.

Index cards are a third design. A small indexer keeps one markdown card per entity (person, project, tool, topic) whose Timeline lists every note that mentions it, with dates, newest first. Cards are rebuilt from the notes on every change, by code, and never call a model, so the same notes always give byte-identical cards. At question time the agent reads the cards the question names, then the source blocks those cards point at.

We measure whether that helps, against the baselines a practitioner would reach for, at an equal context budget. Contributions:

- A benchmark of memory-dependent questions over synthetic markdown vaults generated from a seeded event log, so every answer and every superseded value is known (§3).
- A comparison of seven context builders and an oracle, scored by code, on a world used for development and a second world used once, plus a budget sweep and a second answering model (§5).
- An agent-loop experiment showing where the index stops helping (§6).

2 Related work

Long-term memory benchmarks. LongMemEval [1] tests chat assistants on 500 questions over long chat histories, including a knowledge-update category; Mastra reported that semantic recall reaches about 75% on those questions [2]. MELT [3] targets long-lived agent memory and scores whether stale information recedes. MoM [4] reports a stale rate, the share of answers that match a superseded value, which we adopt. These use chat transcripts rather than a user's notes, and they do not hold the context budget fixed across systems.

Memory for coding agents. Claude Code loads a user-written `CLAUDE.md` and an auto memory file the model writes itself [5]. AMB [6] evaluates memory layers inside coding tasks with executable tests, with conditions for stale and superseded memories. We study the retrieval question underneath: given a user's notes and a fixed budget, what should the agent read?

What is new. We know of no benchmark that compares grep, BM25, embeddings, hybrid retrieval, an agent-written memory file and a deterministic entity index on the same markdown corpus at the same token budget, with superseded values recorded so stale answers are counted rather than estimated.

3 Benchmark

World. A seeded generator creates 90 fictional people and 30 projects and a 26-week event log. The log holds first meetings (role, employer, city, who made the introduction), project kickoffs (owner, language, host, deadline, collaborators), deadline slips, language rewrites, owner handoffs, job changes, moves, personal details, and facts written down wrongly and corrected weeks later. Projects also get a status mention every 5 to 12 days that repeats whichever deadline, language or owner held that day, so an old value is usually written several times before it changes. Daily notes name people by full name, first name or nickname; a first name two people share is never used alone. Sixty of the 90 people have a person note; thirty appear only in daily notes.

Vault. A model writes each note's prose from its fact plan; code rejects a note unless every planted fact's required strings appear verbatim, and retries. A vault has 126 daily notes in a `**HH:MM Title**` block format, 60 person notes, 30 project notes (a third log later changes) and 30 unrelated reference notes: 251 notes and about 2,350 blocks. Vaults are committed, so results do not depend on regenerating them.

Questions. Derived from the log: single facts, updates (the current value of something that changed), multihop (two facts in two notes, such as the city of a project's current owner), aggregate lists, temporal (dates of first meeting or latest mention) and abstention (never stated). Each records its answer keys and, where a value changed, the superseded values. A hash of the id puts 30% in a development split and 70% in a test split.

Two worlds. Seed 7 was the development world. The first retriever (v1) was frozen before its test run; two later changes (§4) were made after seeing seed-7 results, so seed 7 counts as

development data for the final retriever. Seed 8 was generated afterwards and every system ran on it once, with no change after; it gives the headline numbers (257 test questions).

Scoring. Code, not a judge. An answer is correct if it contains every key (a multi-word key also matches when all its content words appear); a list answer must name every member and no other known person; an abstention question is correct only when the answer is "unknown". A wrong answer containing a superseded value is a stale error; "unknown" on an answerable question is an abstain error. We report Wilson 95% intervals and paired bootstrap tests over questions. Two grader fixes were made during development (an answer that names a project's owner as the owner is not an extra list member; "Beta" matches "in beta"), applied to every system.

4 Systems

Every system produces at most B tokens of context (characters / 4) for the same answering model (Claude Sonnet, called through the Claude Code CLI with no tools) and the same prompt. The prompt gives today's date, asks for one line, and says to trust the most recent dated source when sources disagree. Chunk-based systems share one chunking: daily notes split at block headers, other notes at `##` headings, each chunk printed with its path and date.

none no context.

grep chunks containing the question's content words, most distinct words first, newer first on ties.

BM25 Okapi BM25 ($k_1 = 1.2$, $b = 0.75$).

RAG cosine similarity of `nomic-embed-text` embeddings, computed locally, top chunks until B.

hybrid reciprocal rank fusion ($k = 60$) of the BM25 and RAG rankings.

memory a Claude Code style memory file. The model reads the vault one week at a time in date order and rewrites a `MEMORY.md` capped at B tokens after each week, editing facts that changed. The file, cut at B, is the context.

cards index-cards. The cards whose names or aliases appear in the question come first, each with a header and Timeline lines. Then the source blocks those lines point at, round-robin across cards up to 60% of B: blocks that share the most of the question's other words, then the defining note, then newest and oldest alternating. Then up to three cards named most often in what was taken (one hop), with their blocks in the same order minus the word ranking. A question that names no card falls back to BM25.

oracle the blocks that hold the planted facts: a ceiling for the answering step, not a system.

How cards got here. v1 took each card's blocks newest first only and scored 78.6% on seed-7 development questions, missing first-meeting and multihop answers that sit in a card's oldest entry. Adding the defining note and newest–oldest alternation gave v1 as frozen (91.7% on development, 89.2% on seed-7 test). A demo then showed v1 ignoring the question's words inside a card, so a "current deadline" question at a small budget missed the block where the deadline moved. Ranking every card's blocks by question words made seed-7 test accuracy worse (85.4%): the words describe the named card, not the hop card. Restricting the ranking to named cards is the final version (90.7% on seed 7). A determinism bug (hop ties broken by hash order) was fixed at the same time. The baselines were not tuned.

Table 1: Accuracy at a 4,000-token budget. Columns up to Stale: seed 8, the fresh held-out world, 257 test questions, Sonnet answering. Abst. is the share of answerable questions answered "unknown"; Stale is the share of changed-fact questions answered with a superseded value. Seed 7 is the first world (development data for the final retriever; v1 was frozen before its test run). Haiku is seed 8 with Haiku answering.

System	Acc.	95% CI	Single	Multihop	Update	Aggr.	Temporal	Abst.	Stale	Seed 7	Haiku
No memory	7.8	5–12	0	0	0	0	0	100.0	0.0	9.0	7.8
grep	73.5	68–79	94	3	99	84	100	26.6	0.0	73.1	66.1
Memory file	63.0	57–69	47	69	71	24	63	29.5	2.3	76.1	56.8
BM25	77.0	72–82	100	10	99	92	100	23.2	1.1	78.0	72.4
Hybrid	79.8	74–84	100	16	100	96	100	21.5	0.0	79.1	75.1
Embedding RAG	82.1	77–86	98	31	100	88	100	17.7	1.1	84.0	77.0
index-cards v1	88.3	84–92	94	69	95	92	95	10.5	1.1	89.2	
index-cards	89.5	85–93	98	70	94	92	100	9.3	1.1	90.7	80.5
Oracle	93.8	90–96	98	80	100	88	100	5.1	0.0	95.5	88.7

5 Results

Accuracy. On seed 8 at $B = 4{,}000$ (Table 1), cards answer 89.5% of test questions (95% CI 85.1–92.7). Embedding RAG answers 82.1% ($p = 0.0027$), hybrid 79.8% ($p = 0.0001$), BM25 77.0%, the memory file 63.0% and grep 73.5%. No memory answers 7.8%, all of it abstention questions; the oracle reaches 93.8%. The frozen v1 retriever scores 88.3% on seed 8, so the result does not rest on the later changes. Seed 7 gives the same ordering (90.7% against 84.0% for RAG). With Haiku answering instead of Sonnet, cards score 80.5% and RAG 77.0% ($p = 0.1478$, so not significant for the smaller model).

Where the gain comes from. Multihop questions: cards 70%, RAG 31%, BM25 10%. A question like "which city does the current owner of Kelp-DB live in?" names the project. Similarity search finds the project's chunks, but nothing in the question points at the owner's first-meeting note, which holds the city. The project's card names the owner in its newest entries, the one-hop step pulls the owner's card, and the owner's oldest entry is the meeting. On single facts, updates and aggregates the strong retrievers are within a few points of each other.

Stale facts are not where systems fail. With a date on every chunk and an instruction to prefer the newest, stale errors at 4,000 tokens are rare for every system (Table 1). The update is almost always inside the window, and the model reads dates well. Most misses are abstentions: the needed note never made it into the context. Stale errors appear when the budget is tight. At 1,000 tokens, 13.8% of cards' answers to changed facts give an old value, because little more than a card's header, its newest lines and its defining note (which holds the initial values) fit.

The memory file keeps what it chose to keep, and how much it keeps varies. On seed 8 it grew past its cap (22,511 characters against a 16,000-character budget) and was cut, the way Claude Code cuts an over-long `MEMORY.md`; it scored 63.0% there against 76.1% on seed 7. It wrote each owner next to their city and does well on multihop (69%), but it dropped collaborators and dates (aggregate 24%, temporal 63%). It also cost 27 model calls of about 15k input tokens each to build; the cards take under a second of CPU and no calls.

Where cards lose. Thirty people have no note and so no card; questions about them fall back to BM25, and on those cards trail embedding retrieval (on seed 8, 85.7% against 95.2% on the 42 such questions, while the two tie at 93.5% on the 92 questions about people with a card). A fallback to the embedding or hybrid ranking is the obvious next change, and we did not make it.

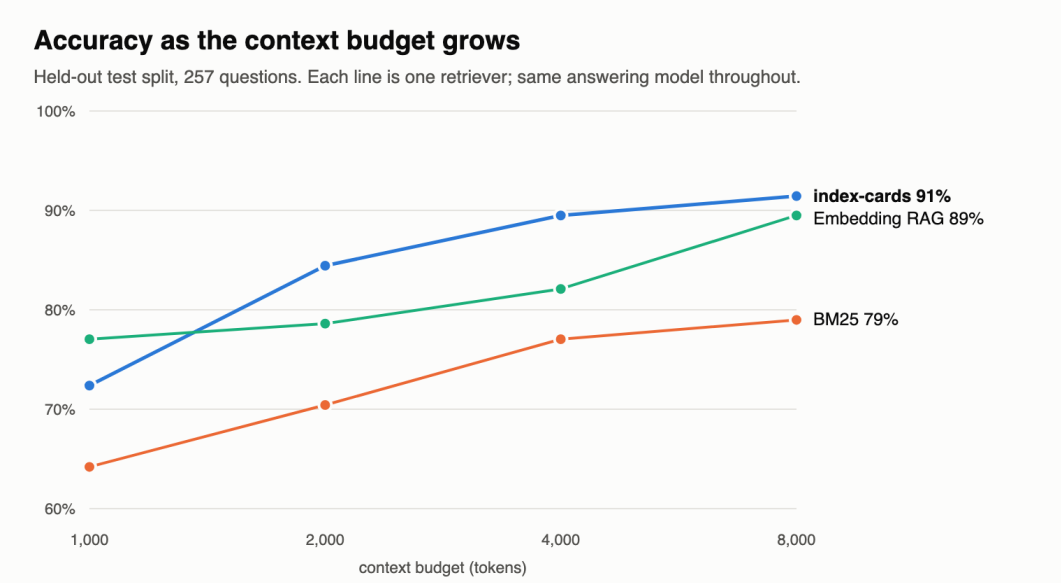


Figure 1: Accuracy against context budget on seed 8 (test split), Sonnet answering.

Table 2: The agent loop: Claude Code with Read, Grep and Glob in the seed-7 vault, no budget, 91 test questions. Tokens are input tokens per question.

	Sonnet			Haiku		
Setup	Acc.	Tokens	Turns	Acc.	Tokens	Turns
Vault only	98.9	20,860	3.9	97.8	24,641	4.6
Vault + memory file	98.9	54,544	4.2	95.6	51,540	4.0
Vault + index-cards	97.8	26,211	5.4	89.0	45,439	6.9

Budget. Figure 1 varies B, and the advantage has a range. At 1,000 tokens RAG is ahead (77.0% against 72.4%): a card's header, Timeline lines and defining note use most of a small budget. At 2,000 and 4,000 tokens cards lead (84.4% against 78.6%, $p = 0.0164$; and the 4,000 result above). At 8,000 tokens the two converge (91.4% against 89.5%, $p = 0.2199$), since RAG's window becomes large enough to hold both notes of a two-hop question. BM25 trails throughout (64.2% to 79.0%).

6 The agent loop

Budgeted retrieval is one way an agent gets memory. The other is to let the agent search. We gave Claude Code Read, Grep and Glob in a copy of the seed-7 vault, no budget and up to 30 turns, under three setups: the vault alone; the vault with the memory file in the system prompt, as Claude Code loads `MEMORY.md`; and the vault with cards plus the one-sentence pointer from the README (Table 2).

Sonnet answers 98.9% from the bare vault with 20,860 input tokens per question; cards add tokens (26,211) and turns and do not add accuracy (97.8%). The memory file doubles the tokens. Haiku does worse with cards (89.0% against 97.8%): most of its extra misses answer "where did I first meet X?" with a date. On a clean vault of this size, a model that can grep freely finds the answer itself, and the index is overhead. Cards earn their place when context is a fixed budget: a hook that injects context, a prompt prefix, a small model, a single retrieval call.

7 Limitations

The vaults are synthetic. A model wrote their prose from fact templates, so they are more regular than real notes, every entity has a distinct name, and no one is referred to only by pronoun or role. Real vaults are larger and messier, which may favour either side: grep and similarity have more noise to wade through, and cards have more aliases to miss. The author of index-cards also built the benchmark; we report each category separately and log every change made after seeing results. A few projects keep receiving status mentions after they were marked cancelled or paused, which makes a handful of status questions ambiguous for every system. The memory baseline simulates Claude Code's auto memory under the same budget and is not the product. The agent loop used one vault and 91 questions.

8 Conclusion

A deterministic entity index is a cheap, inspectable memory layer that matches similarity retrieval on single facts and updates and clearly beats it on questions that need two notes, replicated on a world used once. Stale facts, the usual worry, were rarely the failure once chunks carry dates; missing notes were. When the agent can search for itself without a budget, the index adds cost without adding accuracy. Everything needed to rerun these numbers is in the repository.

References

[1] D. Wu et al. LongMemEval: Benchmarking chat assistants on long-term interactive memory. arXiv:2410.10813, 2024.

[2] Mastra. Yes, you can use RAG for agent memory. https://mastra.ai/research/use-rag-for-agent-memory, 2025.

[3] Shisa.AI. MELT: Testing long-lived memory for agentic AI. https://blog.shisa.ai/posts/melt/, 2026.

[4] MoM: Memory of memory. arXiv:2609.25054, 2026.

[5] Anthropic. Claude Code documentation: memory. https://docs.claude.com/en/docs/claude-code/memory, 2026.

[6] AMB: an open, preregistered benchmark for memory layers in coding agents. https://discuss.huggingface.co/t/179762, 2026.